Design and Instruction-Level Verification of Multi-Cycle and Pipelined RISC-V Processors on an Artix-7 FPGA

Saman Fuyal
Department of Electrical and Electronics Engineering
Kathmandu University
Dhulikhel, Nepal
smnfyl@gmail.com

***Abstract*—Two processors for the RV32I base instruction set, with machine-mode control and status registers, a timer interrupt and the FENCE.I instruction, were written in Verilog-2001 for a Digilent Nexys A7-50T board with an Artix-7 FPGA. One is a multi-cycle design and the other a five-stage pipeline. They share the decoder, ALU, register file, load/store unit and CSR unit and run in the same system-on-chip, so the measured differences come from the microarchitecture alone. Each core reports every retired instruction, and every test, including 100 random instruction streams with traps and timer interrupts per core, is checked instruction by instruction against a Python reference model. The regression of 854 runs on Verilator 5.052 and Icarus Verilog 12.0 passed, and it detected 16 of 16 deliberately injected bugs. A hardware self-test loads random programs onto the board over the serial port and compares every byte the board prints with the reference model. In RTL simulation the pipeline reaches 0.983 DMIPS/MHz on Dhrystone and 0.973 CoreMark/MHz, against 0.396 and 0.448 for the multi-cycle core.**

***Index Terms*—RISC-V, FPGA, pipelined processor, multi-cycle processor, co-simulation, processor verification**

I. Introduction

Computer organisation courses usually build a RISC-V processor in stages: a single-cycle datapath, then a multi-cycle version, then a five-stage pipeline with forwarding and hazard detection [1], [2]. Putting those designs on an FPGA raises questions the textbooks leave open. Block RAM in 7-series devices reads synchronously [3], so the single-cycle organisation does not work with any useful amount of memory. A processor that has only run hand-written test programs may still hide bugs in paths that are rarely used, such as a trap during a stall or a write to a counter register.

Small open-source cores each report a different metric on a different system, and few of them say how correctness was checked instruction by instruction. Larger cores that do check every instruction, such as Ibex with its Spike co-simulation [4], are beyond a one-person project. This paper describes a small-scale version of that discipline. Its contributions are:

- a multi-cycle and a five-stage RV32I core that share every functional unit and differ only in sequencing and hazard handling, so cycles per instruction (CPI), area and timing compare directly;
- a retirement trace and a Python instruction-set simulator (ISS) that check both cores in lockstep on every test layer, including traps, interrupts and random programs;
- an on-board self-test that compares the board's serial output with the ISS byte for byte;
- a flow in which every number in this paper is regenerated from tool output.

II. Related Work

Table I lists five open-source RISC-V cores that target FPGAs. The figures are copied from each project's documentation at the commit given in the references and were not measured again here, so they come from different systems, compilers and settings. The projects also differ in how they treat bad input. SERV's documentation warns against illegal instructions because its decoder assumes they never occur [5]. The cores described here trap on every unsupported or malformed instruction with a defined cause. The usual compliance suites are riscv-tests [6], which report pass or fail by themselves, and riscv-arch-test [7], which compares a memory signature with a reference. The retirement port of Section III-E follows the RVFI interface of riscv-formal [8].

TABLE I. Published Figures of Small RISC-V Cores

Core	Organisation	Performance	Area
PicoRV32 [9]	RV32IMC, multi-cycle	0.516 DMIPS/MHz	750-2000 LUT
SERV [5]	RV32I, bit-serial	not stated	125 LUT, 164 FF
VexRiscv [10]	RV32I, 2-5 stages	0.52-0.82 DMIPS/MHz	504-816 LUT
DarkRISCV [11]	RV32I, 2-3 stages	IPC about 0.7	850-1500 LUT
Ibex [4]	RV32IMC, 2-3 stages	0.904-3.13 CoreMark/MHz	16.9-66.0 kGE
This work	RV32I, multi-cycle and 5-stage	0.396 / 0.983 DMIPS/MHz	1836 / 2083 LUT

Published values, not re-measured. This work: RTL simulation and Yosys estimate for the core alone.

III. Architecture

A. System-on-Chip

Fig. 1 shows the system. The core has one port for instruction fetch and one for data. A bus decoder connects them to a 2 KB boot ROM, a 64 KB RAM and the peripherals. The RAM is a true dual-port block RAM with byte write enables [3], so fetch and data accesses never compete, and every access completes in one cycle. Decoding is strict: an access outside the mapped regions, a store to the ROM, a fetch from the I/O region or a sub-word I/O access returns a fault, so a wild pointer traps instead of reaching a valid address. The peripherals are a UART at 115200 baud, a switch, LED and button port with 10 ms debouncing, an eight-digit seven-segment driver, a machine timer and a system-control block whose TOHOST register ends a simulation with a pass or fail

code. Pins and I/O standards come from the Digilent constraint file [12] and the board manual [13].

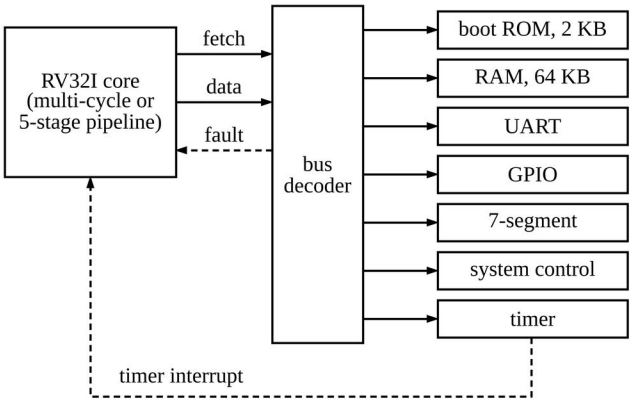


Fig. 1. System-on-chip. Both cores use the same system; all blocks run on one 50 MHz clock.

B. Instruction Set and Traps

Both cores implement RV32I [14], FENCE.I and the machine-mode registers of the privileged specification [15] that a bare-metal program needs: `mstatus`, `mie`, `mip`, `mtvec`, `mscratch`, `mepc`, `mcause`, `mtval`, the 64-bit `mcycle` and `minstret` counters and the read-only identification registers. Exceptions are precise. The cores raise instruction-address-misaligned, access-fault, illegal-instruction, breakpoint, misaligned load and store, and environment-call traps, and take the machine timer interrupt. There is a single privilege level, FENCE and WFI execute as no-ops, and `mtvec` supports direct mode only. Misaligned loads and stores trap instead of being split, which keeps the load/store unit at one access per instruction. A write to `minstret` suppresses that instruction's own increment, as the specification requires. Table II lists the causes and what `mtval` holds for each.

TABLE II. TRAP CAUSES AND MTVAL

mcause	Cause	mtval
0	Instruction address misaligned (jump or branch target)	target
1	Instruction access fault	fetch address
2	Illegal instruction, unknown CSR, write to a read-only CSR	instruction
3	Breakpoint (EBREAK)	pc
4 / 6	Load / store address misaligned	address
5 / 7	Load / store access fault	address
11	Environment call from machine mode (ECALL)	0
2^{31} + 7	Machine timer interrupt	0

C. Multi-Cycle Core

The multi-cycle core (Fig. 2) steps through the states of Fig. 3. FETCH sends the program counter to memory. DECODE receives the instruction one cycle later, because block RAM is synchronous, and reads the register file. EXEC computes the result, the branch decision or the address and retires every instruction except loads, which spend one more cycle in MEM. The core therefore needs three cycles per instruction and four per load. Every exception is detected in EXEC before any state is written, and a pending interrupt is taken in FETCH before the next instruction starts.

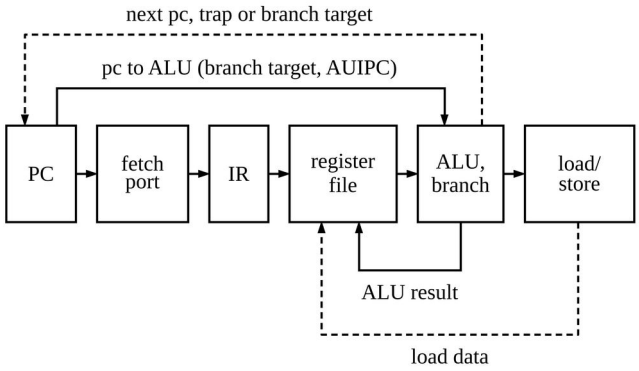


Fig. 2. Multi-cycle datapath. One instruction is in flight; the instruction register (IR) holds it from DECODE to the end of EXEC or MEM.

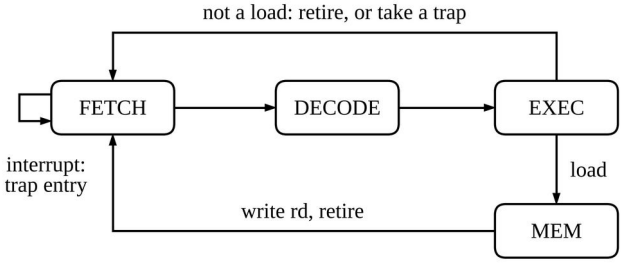


Fig. 3. Control states of the multi-cycle core.

D. Five-Stage Pipeline

The pipeline in Fig. 4 has the classic IF, ID, EX, MEM and WB stages [1], [2]. Results are forwarded from MEM and WB to the ALU inputs in EX, and a value written in WB passes straight through the register file to a read in ID in the same cycle. Two cases still stall for one cycle: a load followed by an instruction that uses its result, and a CSR read followed by a use. Branches and jumps resolve in EX, so a taken branch flushes the two younger instructions and costs two cycles; every branch is predicted not taken.

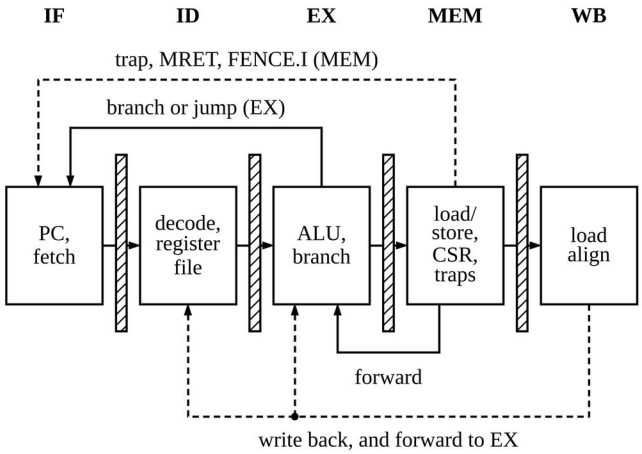


Fig. 4. Five-stage pipeline. Hatched bars are pipeline registers. The hazard unit, which stalls IF and ID for one cycle on a load-use or CSR-use pair, is not drawn.

Fig. 5 shows both penalties. The `add` needs `x5` from the load, so it waits one cycle in ID and then receives the value from WB through the forwarding path. The `beq` is resolved in EX in cycle 6, and the two instructions fetched after it are discarded in cycle 7, when the fetch restarts at the target.

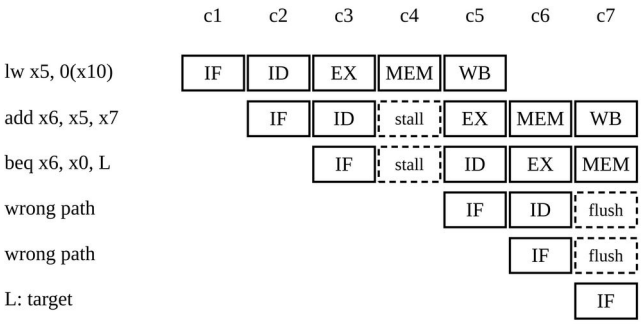


Fig. 5. Pipeline timing of a load-use pair followed by a taken branch. Dashed cells are lost cycles.

Traps, MRET and FENCE.I are handled in MEM, which acts as the commit point. When an instruction reaches MEM with an exception, the instructions behind it are flushed and the older instruction in WB completes, which gives precise exceptions without a reorder buffer. An interrupt is taken on the instruction in MEM in the same way.

Two choices here trade speed for simplicity. Resolving branches in EX instead of ID costs one extra cycle per taken branch, but it keeps the comparator behind the forwarding multiplexers, where the operands are already correct, and avoids a second set of forwarding paths into ID. Committing traps in MEM instead of EX lets a load or store report its access fault in the same stage that performs the access. During a stall the fetch request is not repeated and the block RAM keeps its output, so the stalled instruction is not lost.

E. Retirement Trace

Each core drives a retirement port in the style of RVFI [8]. In the cycle an instruction completes, the port reports the program counter, the instruction word, the destination register and its value, and any memory access; a trap is reported with `mcause`, `mepc` and `mtval`. The testbench prints one line per instruction in the same format as the ISS. The port has no effect on the logic and is removed by synthesis in the FPGA build.

IV. Design Method

A. One Parameter File

`params.json` holds the clock, memory map, peripheral settings and CSR list. A generator writes the Verilog and C headers, the linker scripts, the clock constraint, the Vivado settings and the host loader's settings from it, and the regression first checks that no generated file has been edited by hand. The boot ROM is also generated: the compiled bootloader becomes a Verilog function with one case item per word, which Vivado maps to block RAM [16], so no module contains an initial block or a file read.

B. Coding Rules and Static Checks

Four rules apply to every module. Logic modules contain only synthesisable code: no delays, initial blocks or loops that cannot be unrolled. Every combinational block and every case statement has a default, so no latch can be inferred. Clocked blocks use only non-blocking assignments and combinational blocks only blocking ones. Each file has the same four sections: ports, local parameters and wires, combinational logic and sequential logic. A Python checker enforces these and four further rules, and it is tested on deliberately broken files, each of which must trigger the rule it breaks. Verilator lint with all warnings enabled [17] must report nothing, and Yosys [18] must find no latch, after first showing that it does find one in a broken example. A fourth check parses the Vivado Tcl scripts and rejects any command that is not core Tcl or part of Vivado 2018.2.

C. Software and FPGA Flow

Programs are compiled with GCC 14.2.0 for `rv32i` [19]. A startup file sets the stack, clears BSS and installs a trap vector; the runtime provides serial output, a small `printf` and drivers for the timer, switches and display. The bootloader in ROM receives a program over the serial port in blocks, each with a CRC-32 that it checks before acknowledging, and then jumps to the entry point; a host script sends the program from Windows or Linux and finds the board's serial port by asking each port for the bootloader's reply. A non-project Vivado script synthesises, places and routes the design for the XC7A50T [20] (part `xc7a50ticsg324-1L`) and writes the timing and utilisation reports together with a summary file that this paper reads. Vivado 2018.2 lists Windows 10 as a supported host but not Windows 11 [21]; the board flow was prepared on Windows 11.

V. Verification

A. Reference Model and Lockstep Check

The reference model is an ISS of about 700 lines of Python that models the same system-on-chip: memory map, faults, CSRs, traps and peripheral registers. Fig. 6 shows how it is used. After each simulation the RTL trace and the model trace are compared record by record, and the first difference stops the test with both records printed. A few values depend on timing the model does not represent, such as UART status, `mtime`, `mcycle` and `mip`; for those reads the model takes the value from the RTL record. The timer interrupt is injected into the model at the instruction where the RTL took it, and the check then confirms that `mepc`, `mcause` and the next fetch agree.

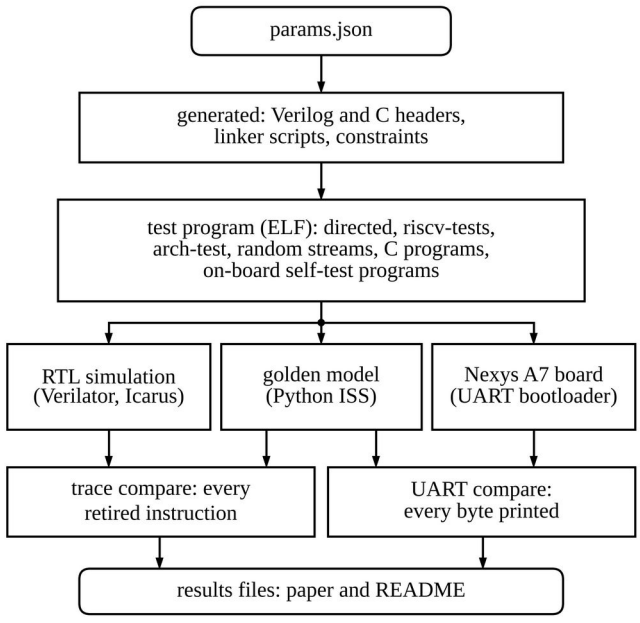


Fig. 6. Verification flow. Simulation results are checked per retired instruction; the board is checked on everything it prints.

B. Test Layers

TABLE III. VERIFICATION LAYERS (LAST FULL REGRESSION)

Layer	What is checked	Passed
static	rule checker, lint, latch proof, Tcl check, generated files	9 / 9
unit	11 modules, model vectors or self-checking	11 / 11
directed	hazards, traps, CSRs, interrupts, decode, FENCE.I	24 / 24
rvtests	riscv-tests rv32ui and rv32mi	232 / 232
arch	riscv-arch-test 2.7.4: I, privileged, Zifencei	220 / 220
random	100 seeds per core, with interrupts	208 / 208
system	C programs at the UART and display pins, bootloader	18 / 18
hil	self-test programs: RTL serial output vs model	128 / 128
docs	diagrams, host tools and board programs	4 / 4
total	Verilator 5.052 and Icarus Verilog 12.0	854 / 854

Every run of the directed, compliance, random and system layers is also compared with the model trace. Runs count both cores. Date: 2026-11-02 14:59:46.

Table III lists the layers. Directed tests target hazards, every trap cause, CSR behaviour, interrupts, memory decoding and FENCE.I. Random programs of about two thousand instructions mix all RV32I and CSR instructions with illegal words, misaligned and unmapped accesses, and timer interrupts at random intervals. System tests run the C programs and read their output from the UART and display pins, so the full top level is tested with its real pins, and one test loads a program through the bootloader at the real baud rate. Four riscv-tests need features that are not implemented, such as misaligned access or debug triggers; they are checked for the exact failure the model also reaches. The long tests run on Verilator; Icarus Verilog [22] runs the same testbench on the shorter layers, which catches constructs one simulator accepts and the other does not.

One deviation from the published suites needs stating. For eight riscv-arch-test branch and jump tests, the reference signatures shipped with release 2.7.4 were produced for a configuration with compressed instructions, in which a target aligned to two bytes is legal. These cores must trap on such a target, so the eight tests are checked against the ISS signature instead. The four riscv-tests with an expected failure are rv32ui-ma_data (misaligned access), rv32mi-breakpoint (debug triggers), rv32mi-pmpaddr (memory protection) and rv32mi-zicntr (user-level counters).

C. Coverage and Fault Injection

Table IV summarises a Verilator coverage build of the regression. It hits 99.7 % of the lines of the pipelined core and 98.4 % of the multi-cycle core, and 98.0 % of all RTL lines. Coverage only shows what was exercised, so the tests were also checked by fault injection: 16 bugs of the kind that are easy to make, such as a missing forwarding path or a wrong trap cause, were inserted one at a time, and the regression caught 16 of 16. Table V lists them with the first test that failed.

TABLE IV. COVERAGE IN THE VERILATOR COVERAGE REGRESSION

Measure	Multi-cycle	Pipeline
Line coverage, core	98.4 %	99.7 %
Toggle coverage, core	82.1 %	86.2 %
Mnemonics retired	47 of 49	47 of 49
Trap causes taken	10 of 10	10 of 10

ECALL and EBREAK never retire because they always trap; they are counted among the trap causes. All RTL lines, both cores and the peripherals: 98.0 %.

TABLE V. INJECTED BUGS AND THE FIRST TEST THAT FAILED

#	Injected bug	First failing test
M01	no MEM to EX forwarding for rs1	directed/csr
M02	no WB to EX forwarding for rs2	directed/csr
M03	load-use interlock removed	directed/hazards
M04	register-file write-through bypass removed	directed/csr
M05	instruction after a taken branch not flushed	directed/hazards
M06	mepc taken from the wrong stage	directed/irq
M07	FENCE.I does not flush	directed/fencei
M08	SRA computed as SRL	unit/alu
M09	BGEU uses the signed compare	unit/alu
M10	SLLI with shamt[5] = 1 accepted	unit/decode
M11	minstret write does not suppress the increment	unit/csr
M12	LH not sign-extended	unit/lsu
M13	stores to the boot ROM do not fault	unit/bus
M14	misaligned jump target not trapped (multi-cycle)	directed/traps
M15	trap entry leaves interrupts enabled	unit/csr
M16	SB always writes byte lane 0	unit/lsu

D. On-Board Self-Test

Simulation cannot show that the bitstream behaves like the RTL. The self-test checks this directly, as the right half of Fig. 6 shows. The host loads one program after another through the bootloader. Each random program, generated without anything whose value depends on timing, ends by printing all registers and 2 KB of data memory as hexadecimal text; each riscv-test prints its result code. Every program then jumps back into the bootloader, so the whole set runs after a single reset, and the host compares every byte with the output of the ISS for the same file. The random programs for the board differ from the simulation ones in three ways: they contain no interrupts, they do not read `mcycle` or `minstret` (on the board these also count the bootloader's instructions), and their faulting loads use unmapped addresses only, because the RAM still holds the previous program. Each one first clears the CSRs it can read. Each character takes 10 bit periods of 434 clock cycles, so the speed of the self-test is set by the serial link. In the regression, 128 of 128 runs of these programs on the RTL produced serial output identical to the model. On the board, 158 of 158 programs matched for the pipelined core, 489628 bytes compared in total.

VI. RESULTS AND DISCUSSION

A. Resources and Timing

TABLE VI. RESOURCES AND TIMING ON THE XC7A50T

Resource	Multi-cycle	Pipeline	Source
LUT, core	1836	2083	Yosys
FF, core	452	977	Yosys
LUT, system	2369	2624	Yosys
FF, system	1122	1554	Yosys
LUT, system	2132	2288	Vivado 2018.2
FF, system	1136	1574	Vivado 2018.2
RAMB36 / RAMB18	16 / 2	16 / 2	Vivado 2018.2
WNS at 50 MHz (ns)	1.459	2.266	Vivado 2018.2
Fmax estimate (MHz)	53.9	56.4	Vivado 2018.2

The XC7A50T has 32,600 LUTs and 65,200 flip-flops [20]. Fmax = 1 / (period - WNS) from the post-route timing report.

Table VI gives the resource use. The Yosys figures are estimates from `synth_xilinx`, which maps to the same primitives as Vivado but optimises differently, so they mainly show the difference between the cores. The pipelined core needs 1.13 times the LUTs and 2.16 times the flip-flops of the multi-cycle core. The extra flip-flops are the four pipeline registers, and most of the extra LUTs are forwarding

multiplexers and hazard logic. Neither core uses a DSP slice, because RV32I has no multiply instruction. Post-route timing at 50 MHz gives a worst setup slack of 2.266 ns for the pipeline and 1.459 ns for the multi-cycle core (positive slack means the clock is met), an estimated Fmax of 56.4 and 53.9 MHz.

B. Cycles per Instruction and Benchmarks

Fig. 7 and Table VII give exact cycle counts from RTL simulation. The benchmark figures come from the programs' own `mcycle` and `minstret` readings around the timed loop. Dhrystone [23] is built with `-O3 -fno-inline` and CoreMark [24] with `-O2 -funroll-loops`. The pipeline is 2.48 times faster per clock on Dhrystone and 2.17 times on CoreMark.

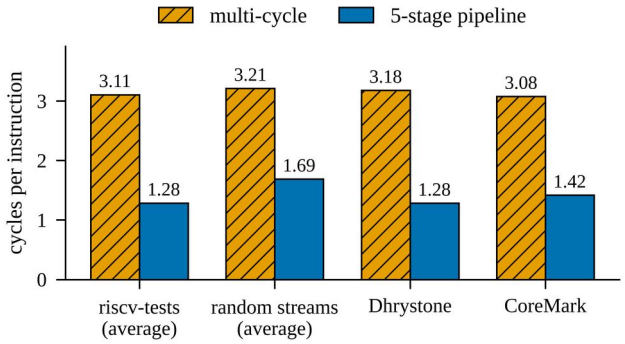


Fig. 7. Cycles per instruction of the two cores (RTL simulation, exact counts). Dhrystone and CoreMark values cover the timed loop only.

TABLE VII. PERFORMANCE PER CLOCK

Measure	Multi-cycle	Pipeline
Dhrystone DMIPS/MHz, simulation	0.396	0.983
CoreMark/MHz, simulation (short run)	0.448	0.973
Timer interrupt latency (cycles)	0-3	0-2
Dhrystone DMIPS/MHz, board	0.396	0.983
CoreMark/MHz, board	0.448	0.973
Self-test programs identical, board	158 of 158	158 of 158

DMIPS uses 1757 Dhrystones per second. The simulated CoreMark run is shorter than the 10 s its rules require; the board run is longer. Interrupt latency counts from the pending bit to the trap.

In the pipeline, CPI = 1 + s + 2f, where s is the number of load-use and CSR stalls per instruction and f the number of taken branches, jumps and other redirects. Over the whole Dhrystone program, including printing and waiting for the UART, s = 0.322 and 2f = 0.654; for CoreMark s = 0.158 and 2f = 0.506. Redirects cost 2.0 to 3.2 times as much as stalls, and 33 % of all Dhrystone cycles are flushed slots, so predicting branches in IF or resolving them in ID is the obvious next step. Of all instructions, 65 % take at least one operand from a forwarding path.

For comparison, PicoRV32 reports 0.516 DMIPS/MHz at about 4.1 cycles per instruction with its multiplier and divider enabled [9], and VexRiscv in its small RV32I configuration reports 0.52 DMIPS/MHz without a bypass and 0.82 with one [10]. The pipeline here reaches 0.983 DMIPS/MHz and the multi-cycle core 0.396, executing about 452 instructions per Dhrystone iteration. The published figures come from other systems and compilers, so they show a range rather than a ranking.

The multi-cycle core takes the timer interrupt exactly 3 cycles after the pending bit rises. The pipeline takes it in 0 to 2 cycles, depending on the stage the next instruction has reached, because it takes the interrupt on the instruction in MEM. In both cases the latency is set by the hardware and does not depend on the program.

C. On the Board

Fig. 8 shows the board running a demonstration program. The board rows of Table VII and the Vivado rows of Table VI are filled from the files the board scripts write.

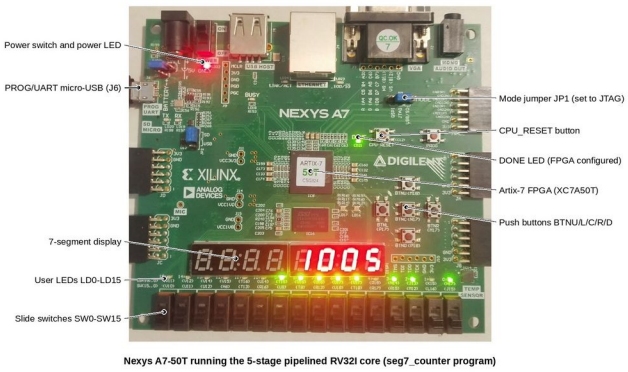


Fig. 8. Nexys A7-50T running the seven-segment counter program.

D. Limitations

- One author wrote both the RTL and the model, so a misreading of the specification could appear in both. The riscv-tests and the riscv-arch-test signatures come from other sources and limit this risk.
- There is no formal proof; the RVFI-style port would allow riscv-formal to be connected.
- The simulated CoreMark run lasts two iterations, so it is not a valid CoreMark score; only the board run, at more than 10 s, gives one.
- Vivado 2018.2 is not supported on Windows 11 [21]. Both builds ran there without problems, but another machine may need Windows 10 or a newer Vivado release.
- The instruction set is RV32I with machine mode only: no multiply, compressed instructions, user mode, memory protection or caches.

VII. CONCLUSION

Two RV32I cores were built from one set of functional units and compared on the same system. The five-stage pipeline lowers CPI from 3.11 to 1.28 on the riscv-tests and is 2.48 times faster per clock on Dhrystone, for 1.13 times the core LUTs in the Yosys estimate. Every simulation was checked one retired instruction at a time against a reference model, the test set caught all 16 injected bugs, and the same model checks the board through its serial output. The next steps are early branch resolution, the M extension on the DSP slices and a formal check through riscv-formal.

REFERENCES

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[2] S. L. Harris and D. Harris, Digital Design and Computer Architecture, RISC-V Edition. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[3] Xilinx, "7 Series FPGAs memory resources user guide," UG473 (v1.14), Jul. 2019.

[4] lowRISC, "Ibex RISC-V core," GitHub repository, commit 1b29dae (README performance table and dv/cosim Spike co-simulation). [Online]. Available: https://github.com/lowRISC/ibex

[5] O. Kindgren, "SERV: the SErial RISC-V CPU," GitHub repository, commit f200eb2. [Online]. Available: https://github.com/olofk/serv

[6] RISC-V International, "riscv-tests," GitHub repository, commit 3df703c. [Online]. Available: https://github.com/riscv-software-src/riscv-tests

[7] RISC-V International, "riscv-arch-test," GitHub repository, release 2.7.4. [Online]. Available: https://github.com/riscv-non-isa/riscv-arch-test

[8] YosysHQ, "riscv-formal: RISC-V formal verification framework," GitHub repository. [Online]. Available: https://github.com/YosysHQ/riscv-formal

[9] C. Wolf, "PicoRV32: a size-optimized RISC-V CPU," GitHub repository, commit ef203c2. [Online]. Available: https://github.com/YosysHQ/picorv32

[10] SpinalHDL, "VexRiscv: a FPGA friendly 32 bit RISC-V CPU implementation," GitHub repository, commit baf7dc8. [Online]. Available: https://github.com/SpinalHDL/VexRiscv

[11] M. Samsoniuk, "DarkRISCV," GitHub repository, commit 9555119. [Online]. Available: https://github.com/darklife/darkriscv

[12] Digilent, "digilent-xdc: master XDC files for Digilent boards," GitHub repository, commit c8644a9. [Online]. Available: https://github.com/Digilent/digilent-xdc

[13] Digilent, "Nexys A7 reference manual." [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[14] A. Waterman and K. Asanović, Eds., "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," RISC-V International, version 20240411, Apr. 2024.

[15] RISC-V International, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture (Machine ISA version 1.13)," version 20240411, Apr. 2024.

[16] Xilinx, "Vivado Design Suite user guide: Synthesis," UG901 (v2019.1), Jun. 2019.

[17] W. Snyder et al., "Verilator," version 5.052. [Online]. Available: https://verilator.org

[18] C. Wolf et al., "Yosys Open SYnthesis Suite," version 0.70. [Online]. Available: https://github.com/YosysHQ/yosys

[19] xPack Project, "xPack GNU RISC-V Embedded GCC," version 14.2.0-3. [Online]. Available: https://xpack-dev-tools.github.io/riscv-none-elf-gcc-xpack/

[20] Xilinx, "7 Series FPGAs data sheet: Overview," DS180 (v2.6.1), Sep. 2020.

[21] Xilinx, "Vivado Design Suite user guide: Release notes, installation, and licensing," UG973 (v2018.2), Jul. 2018.

[22] S. Williams, "Icarus Verilog," version 12.0. [Online]. Available: https://github.com/steveicarus/iverilog

[23] R. P. Weicker, "Dhrystone: A synthetic systems programming benchmark," Commun. ACM, vol. 27, no. 10, pp. 1013-1030, Oct. 1984, doi: 10.1145/358274.358283.

[24] EEMBC, "CoreMark," GitHub repository, commit 1f483d5. [Online]. Available: https://github.com/eembc/coremark